Reproducing and Extending Learning to Draft

Adaptive Speculative Decoding with Reinforcement Learning

Zifan Ying Shaoshao Xiong

CS498 Machine Learning Systems

May 2026

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

Why Speculative Decoding?

Inference bottleneck

LLM generation is autoregressive: each token depends on the previous token, so decoding underuses GPU parallelism.

Speculative decoding

A small draft model proposes tokens; the target model verifies them in parallel.

The control problem

Draft more tokens and deeper trees can increase accepted tokens, but they also add latency. The best choice varies by context, model, precision, and hardware.

Learning to Draft

- LTD replaces static EAGLE-3 control with two PPO policies.
- The size policy chooses how many draft candidates to verify.
- The depth policy chooses whether to keep expanding the draft tree.
- Reward is based on throughput, not acceptance length alone.

Core question

When do learned adaptive policies beat a strong static EAGLE-3 baseline, and when do they transfer?

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

Reproduction Scope

Models

- Llama-3.1-8B-Instruct
- DeepSeek-R1-Distill-Llama-8B
- Qwen3-14B
- Vicuna-13B-v1.3
- Qwen3-32B, 8-bit

Benchmarks

11 datasets: Alpaca, GSM8K, HumanEval, MT-Bench, QA, summarization, Math, LeetCode, MMLU, BBH, and TheoremQA.

Metric

Throughput speedup over autoregressive decoding, plus Eagle3+RL / Eagle3 for policy gain.

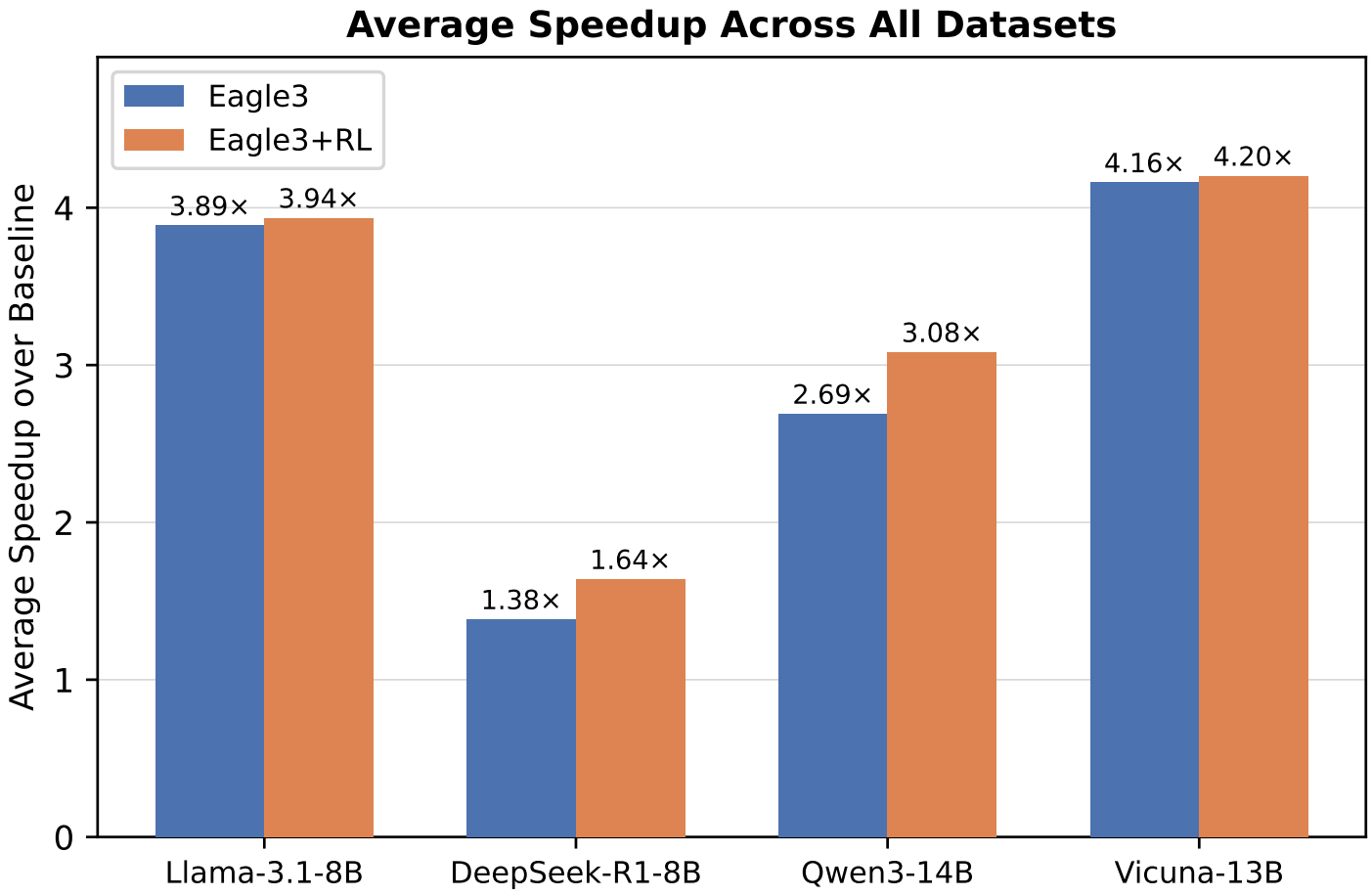
Extension Experiments

- Qwen3-32B reproduction: complete the fifth model using 8-bit quantization.
- Quantization transfer: train Qwen3-14B policies under bf16, int4, or int8; evaluate across all three precisions.
- Hardware transfer: train bf16 Qwen3-14B policies on H100 or A5090; deploy on RTX A6000.
- Naive model parallelism: split Qwen3-32B across two GPUs with automatic layer sharding.

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

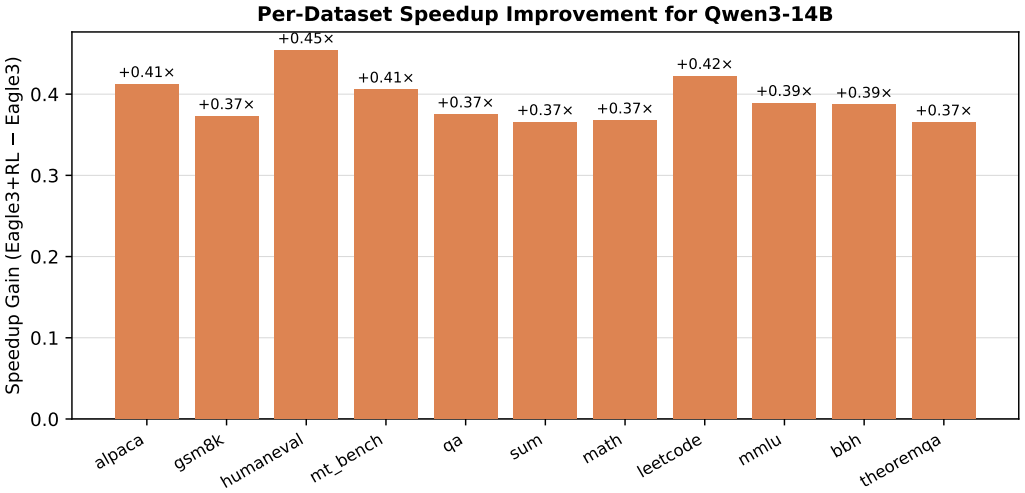
Initial Four-Model Reproduction



Takeaway

Eagle3+RL improves all four initial models, with the largest gains on DeepSeek-R1-8B and Qwen3-14B.

Qwen3-14B: Strong Cross-Dataset Transfer



Average speedup

- Eagle3: 2.69×
- Eagle3+RL: 3.08×
- Relative gain over Eagle3: 14.5%

Key point

The controller was trained on HumanEval, but every benchmark shows a positive gain.

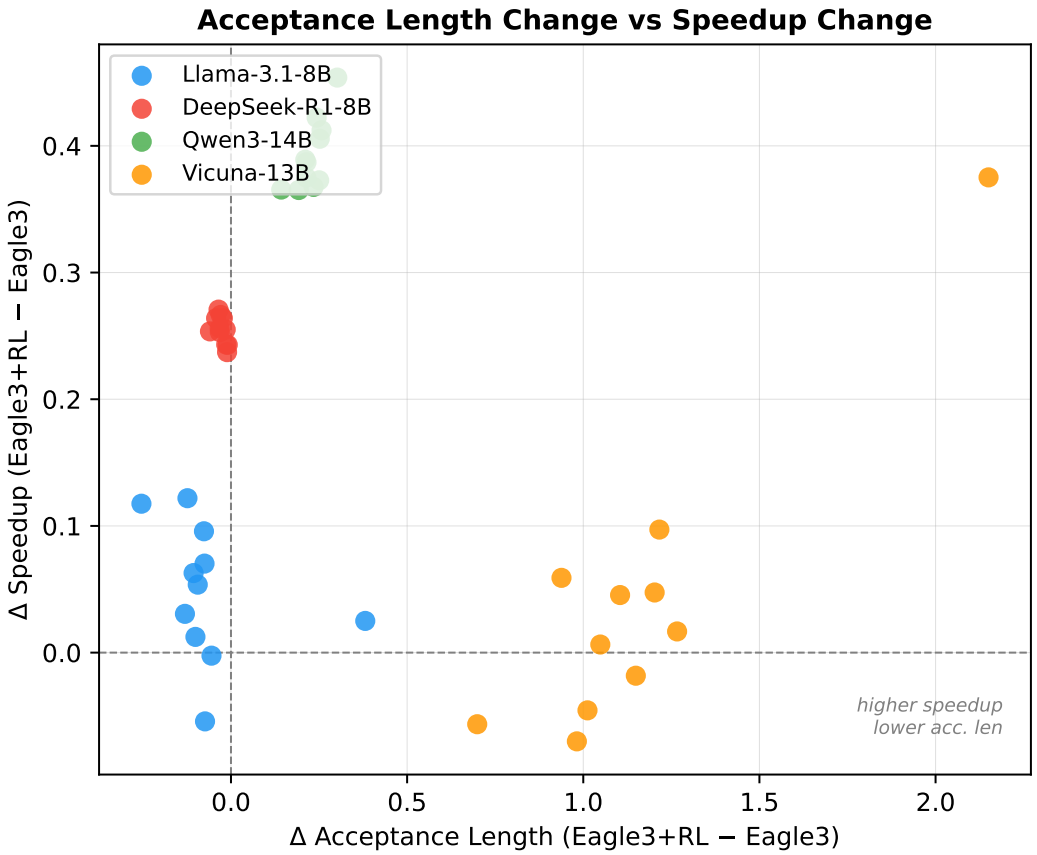
Qwen3-32B: Fifth Model Reproduced

Dataset	Eagle3	Eagle3+RL	RL/Eagle3
Alpaca	2.99×	3.19×	1.069
GSM8K	3.29×	3.45×	1.047
HumanEval	2.93×	3.14×	1.070
Math	3.36×	3.60×	1.070
TheoremQA	3.15×	3.41×	1.083
Sum	2.49×	2.46×	0.992
Average, all 11	3.00×	3.16×	1.054

Takeaway

Qwen3-32B under 8-bit quantization benefits from LTD on 10 of 11 datasets, but the average gain is smaller than Qwen3-14B.

Throughput Beats Acceptance-Length Maximization



Observation

Some points improve speedup even when acceptance length decreases.

Interpretation

The RL policies are optimizing latency-aware throughput, not simply accepted tokens per speculative step.

Roadmap

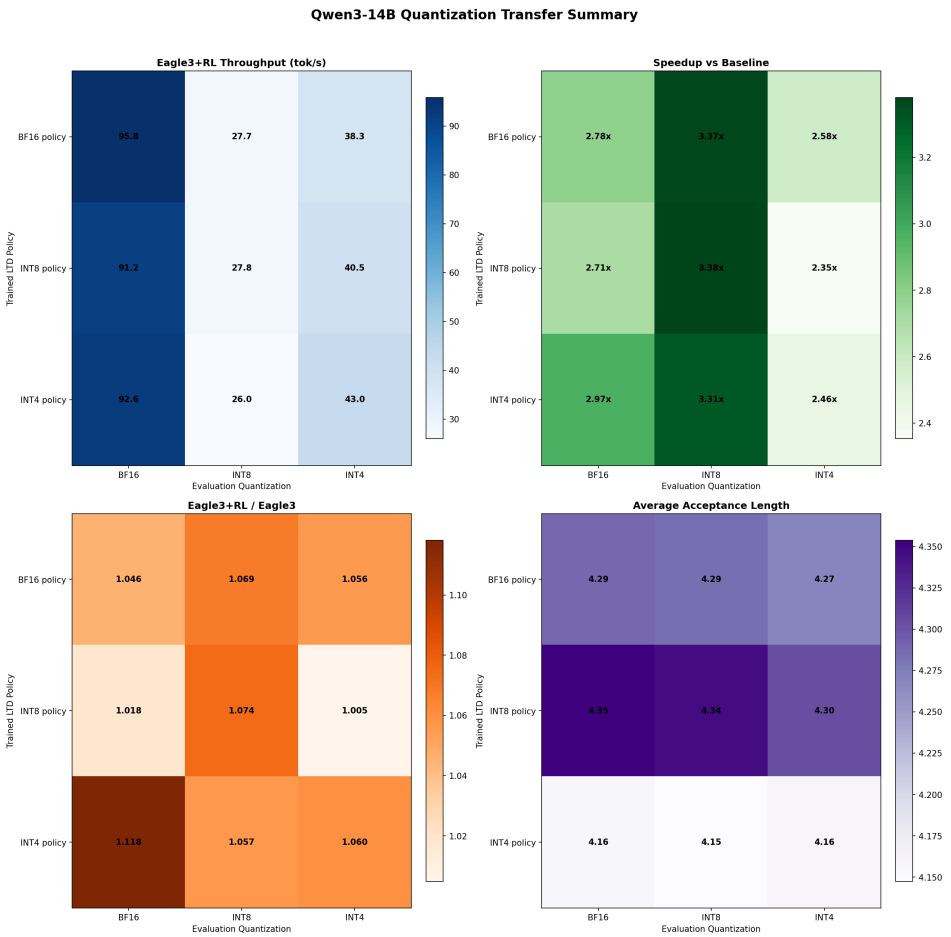
1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

Quantization Transfer Is Mostly Robust

Policy	bf16	int4	int8	Avg.
bf16	1.046	1.056	1.069	1.057
int4	1.118	1.060	1.057	1.078
int8	1.018	1.005	1.074	1.032

Summary

Average RL/Eagle3 is 1.056 across 36 policy-precision-dataset conditions; only two individual runs regress.

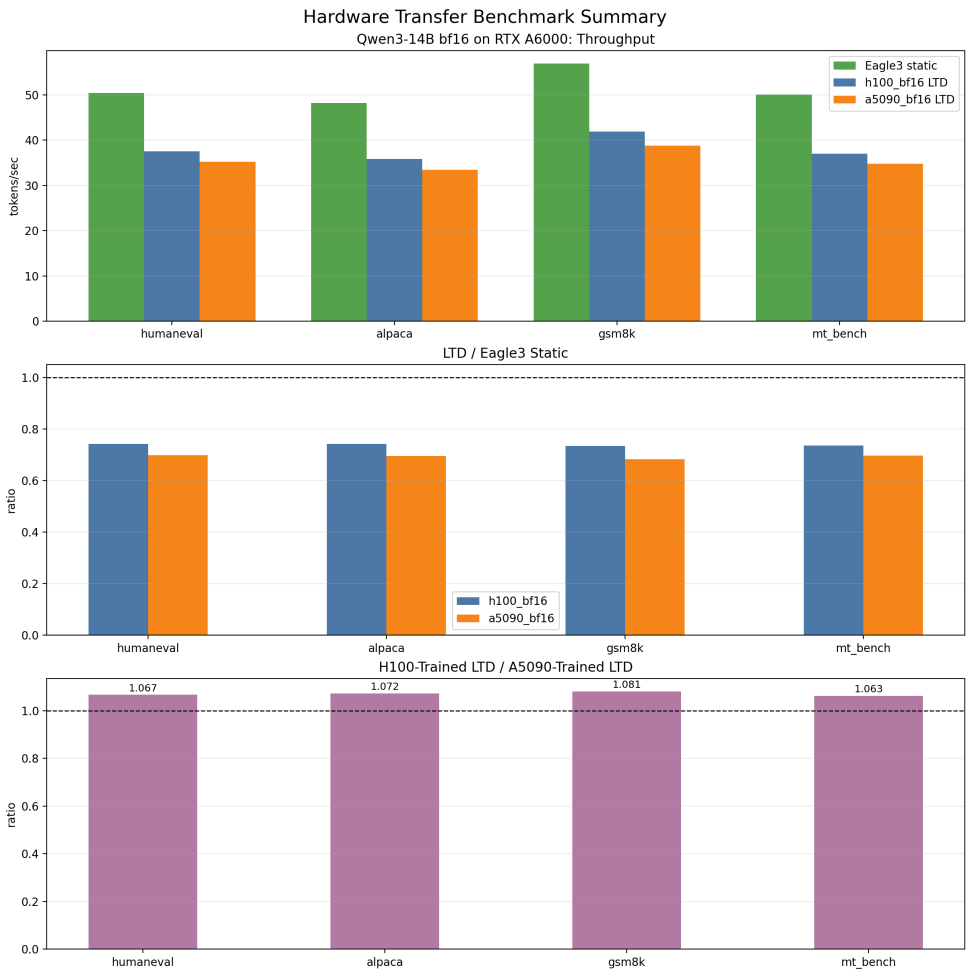


Hardware Transfer Is Brittle

Train GPU	Eagle3	RL	RL/E3
A5090	2.55×	1.76×	0.693
H100	2.55×	1.89×	0.739

Takeaway

Policies trained on H100 or A5090 are still faster than autoregressive decoding, but fail to beat static Eagle3 on RTX A6000.



Why Hardware Transfer Fails

- The reward learns a hardware-specific latency surface.
- Larger verification batches are amortized differently across GPUs.
- Memory bandwidth and kernel overhead shift the best size-depth trade-off.
- A policy that is optimal on the training GPU can choose actions that are too costly on the target GPU.

Implication

Quantization transfer looks feasible, but hardware transfer likely needs hardware-conditioned observations or lightweight target-GPU fine-tuning.

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

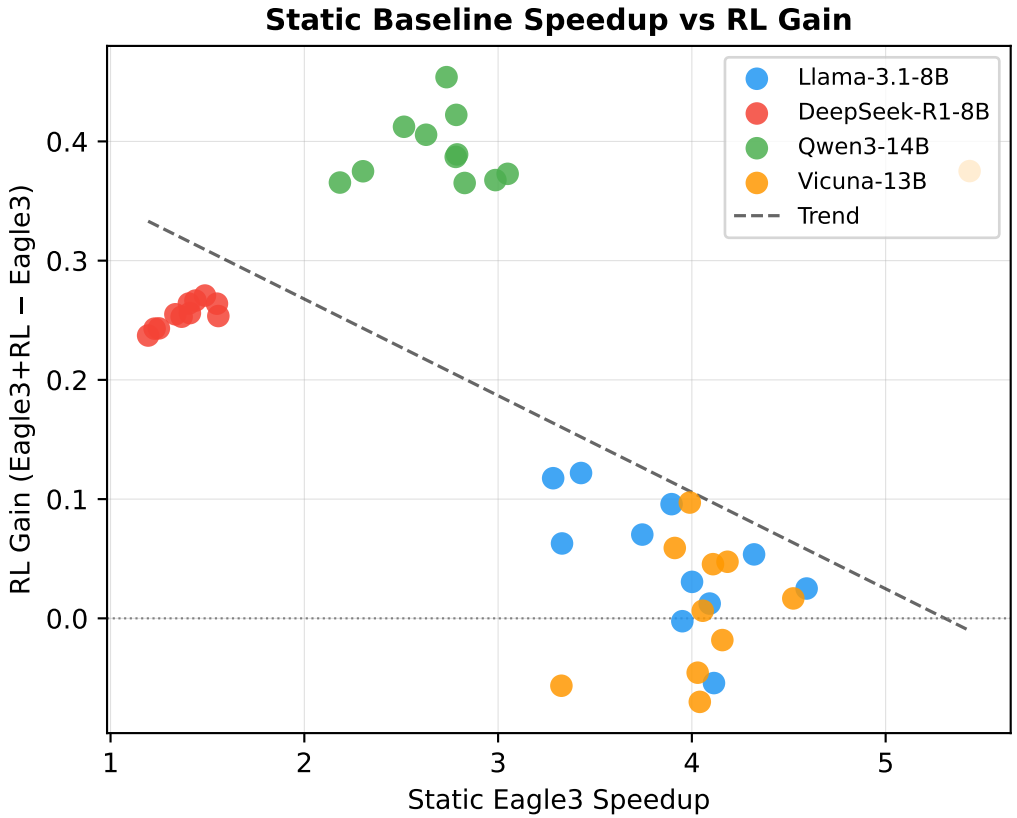
Naive Two-GPU Model Parallelism

Setting	Base tp	E3 tp	RL tp	RL speedup	RL/E3
1 GPU	5.60	16.75	17.67	3.16×	1.054
2 GPUs	5.11	15.36	16.19	3.17×	1.053

Takeaway

The relative LTD gain is preserved under layer sharding, but absolute throughput drops by about 8%. Naive model parallelism helps placement more than latency.

Saturation Explains Small Gains



Pattern

Stronger static Eagle3 baselines leave less room for RL improvement.

Examples

- Vicuna and Llama: static baseline already strong.
- DeepSeek and Qwen3-14B: larger adaptive-control gains.
- Qwen3-32B: positive but modest under 8-bit.

Roadmap

1. Motivation
2. Setup
3. Main Results
4. Transfer
5. Deployment
6. Conclusion

Main Takeaways

1. We reproduced LTD on five target models, including Qwen3-32B under 8-bit quantization.
2. LTD improves same-hardware, same-precision EAGLE-3 throughput in most settings.
3. Quantization transfer is mostly robust across bf16, int4, and int8.
4. Hardware transfer is brittle: throughput-optimized policies encode GPU-specific trade-offs.
5. Naive two-GPU sharding preserves relative LTD gains but does not improve absolute throughput.

Next Steps

- Evaluate temperature sampling, not only greedy decoding.
- Analyze policy action distributions across tasks and deployment conditions.
- Add hardware-conditioned features or lightweight fine-tuning for target GPUs.
- Explore true distributed speculative decoding with pipeline or tensor parallelism.

Questions?